//Config file:-

```php
<?php
/**
* Main Configuration File
* AgroPest POS System
*/

// Start session if not already started
if (session_status() === PHP_SESSION_NONE) {
session_start();
}

// Error Reporting (Set to 0 in production)
error_reporting(E_ALL);
ini_set('display_errors', 1);

// Timezone
date_default_timezone_set('Africa/Nairobi');

// Site Configuration
define('SITE_NAME', 'AgroPest POS');
define('SITE_URL', 'http://localhost/agropest_pos/');
define('ADMIN_URL', SITE_URL . 'admin/');
define('POS_URL', SITE_URL . 'pos/');

// Directory Paths
define('ROOT_PATH', dirname(__DIR__) . '/');
define('UPLOAD_PATH', ROOT_PATH . 'assets/uploads/');
define('PRODUCT_IMAGE_PATH', UPLOAD_PATH . 'products/');

// URL Paths
define('ASSETS_URL', SITE_URL . 'assets/');
define('CSS_URL', ASSETS_URL . 'css/');
define('JS_URL', ASSETS_URL . 'js/');
define('IMG_URL', ASSETS_URL . 'images/');

// Business Settings
define('BUSINESS_NAME', 'AgroPest Agrovet');
define('BUSINESS_PHONE', '+254 700 000 000');
define('BUSINESS_EMAIL', 'info@agropest.co.ke');
define('BUSINESS_ADDRESS', 'Nairobi, Kenya');
define('VAT_RATE', 16); // Kenya VAT Rate
define('CURRENCY', 'KES');
//define('CURRENCY_SYMBOL', 'Ksh');

// Session timeout (in seconds) - 2 hours
define('SESSION_TIMEOUT', 7200);

// Pagination
define('RECORDS_PER_PAGE', 20);
```

```
// File Upload Settings
define('MAX_FILE_SIZE', 5242880); // 5MB in bytes
define('ALLOWED_IMAGE_TYPES', ['image/jpeg', 'image/png', 'image/gif']);

// Include database connection
require_once ROOT_PATH . 'config/database.php';

// Include core classes
require_once ROOT_PATH . 'classes/Database.php';
require_once ROOT_PATH . 'classes/Auth.php';
require_once ROOT_PATH . 'classes/User.php';
require_once ROOT_PATH . 'classes/Category.php';
require_once ROOT_PATH . 'classes/Product.php';
require_once ROOT_PATH . 'classes/Stock.php';
require_once ROOT_PATH . 'classes/Supplier.php';

// Helper Functions
require_once ROOT_PATH . 'includes/functions.php';

// Check session timeout
if (isset($_SESSION['last_activity']) && (time() - $_SESSION['last_activity'] > SESSION_TIMEOUT)) {
session_unset();
session_destroy();
header("Location: " . SITE_URL . "index.php?timeout=1");
exit();
}
$_SESSION['last_activity'] = time();
?>
```

// Auth Class file

```
<?php
/**
* Authentication Class
* Handles user login, logout, and session management
*/

class Auth {
private $db;
public function __construct() {
$this->db = new Database();
}
/**
* Login user
*/
public function login($username, $password) {
// Get user by username
$sql = "SELECT u.*, up.* FROM users u
LEFT JOIN user_permissions up ON u.user_id = up.user_id
WHERE u.username = :username AND u.status = 'active' LIMIT 1";
$user = $this->db->fetchOne($sql, ['username' => $username]);
```

```php
if ($user && password_verify($password, $user['password_hash'])) {
// Set session variables
$_SESSION['user_id'] = $user['user_id'];
$_SESSION['username'] = $user['username'];
$_SESSION['full_name'] = $user['full_name'];
$_SESSION['role'] = $user['role'];
$_SESSION['email'] = $user['email'];
$_SESSION['logged_in'] = true;
// Store permissions
$_SESSION['permissions'] = [
'can_add_products' => $user['can_add_products'] ?? false,
'can_edit_products' => $user['can_edit_products'] ?? false,
'can_delete_products' => $user['can_delete_products'] ?? false,
'can_manage_stock' => $user['can_manage_stock'] ?? false,
'can_make_sales' => $user['can_make_sales'] ?? true,
'can_view_reports' => $user['can_view_reports'] ?? false,
'can_manage_users' => $user['can_manage_users'] ?? false,
'can_manage_customers' => $user['can_manage_customers'] ?? false,
'can_process_refunds' => $user['can_process_refunds'] ?? false,
'can_give_discounts' => $user['can_give_discounts'] ?? false,
'max_discount_percent' => $user['max_discount_percent'] ?? 0
];
// Update last login
$this->db->update('users',
['last_login' => date('Y-m-d H:i:s')],
'user_id = :user_id',
['user_id' => $user['user_id']]
);
// Log activity
$this->logActivity($user['user_id'], 'login', 'User logged in');
return true;
}
return false;
}
/**
* Logout user
*/
public function logout() {
if (isset($_SESSION['user_id'])) {
$this->logActivity($_SESSION['user_id'], 'logout', 'User logged out');
}
session_unset();
session_destroy();
return true;
}
/**
* Check if user is logged in
*/
public function isLoggedIn() {
return isset($_SESSION['logged_in']) && $_SESSION['logged_in'] === true;
```

```
}
/**
* Check if user is admin
*/
public function isAdmin() {
return isset($_SESSION['role']) && $_SESSION['role'] === 'admin';
}
/**
* Check if user is cashier
*/
public function isCashier() {
return isset($_SESSION['role']) && $_SESSION['role'] === 'cashier';
}
/**
* Check user permission
*/
public function hasPermission($permission) {
if ($this->isAdmin()) {
return true; // Admin has all permissions
}
return isset($_SESSION['permissions'][$permission]) &&
$_SESSION['permissions'][$permission] == true;
}
/**
* Require login
*/
public function requireLogin() {
if (!$this->isLoggedIn()) {
header("Location: " . SITE_URL . "index.php");
exit();
}
}
/**
* Require admin role
*/
public function requireAdmin() {
$this->requireLogin();
if (!$this->isAdmin()) {
header("Location: " . SITE_URL . "dashboard-redirect.php");
exit();
}
}
/**
* Get current user ID
*/
public function getUserId() {
return $_SESSION['user_id'] ?? null;
}
/**
* Get current user data
```

```
*/
public function getUser() {
if (!$this->isLoggedIn()) {
return null;
}
$sql = "SELECT * FROM users WHERE user_id = :user_id";
return $this->db->fetchOne($sql, ['user_id' => $this->getUserId()]);
}
/**
* Log user activity
*/
private function logActivity($userId, $type, $description) {
$data = [
'user_id' => $userId,
'activity_type' => $type,
'activity_description' => $description,
'ip_address' => $_SERVER['REMOTE_ADDR'] ?? 'Unknown',
'user_agent' => $_SERVER['HTTP_USER_AGENT'] ?? 'Unknown'
];
$this->db->insert('activity_log', $data);
}
/**
* Change password
*/
public function changePassword($userId, $newPassword) {
$passwordHash = password_hash($newPassword, PASSWORD_DEFAULT);
return $this->db->update('users',
['password_hash' => $passwordHash],
'user_id = :user_id',
['user_id' => $userId]
);
}
}
?>
```

// User Class file

```
<?php
/**
* User Class
* Handles all user-related operations
*/

class User {
private $db;
public function __construct() {
$this->db = new Database();
}
/**
* Get all users
*/
```

```
public function getAllUsers($status = null) {
$sql = "SELECT u.*,
(SELECT COUNT(*) FROM sales WHERE served_by = u.user_id) as total_sales
FROM users u";
$params = [];
if ($status) {
$sql .= " WHERE u.status = ?";
$params[] = $status;
}
$sql .= " ORDER BY u.created_at DESC";
return $this->db->fetchAll($sql, $params);
}
/**
* Get user by ID
*/
public function getUserById($userId) {
$sql = "SELECT u.*, up.*
FROM users u
LEFT JOIN user_permissions up ON u.user_id = up.user_id
WHERE u.user_id = ?";
return $this->db->fetchOne($sql, [$userId]);
}
/**
* Get user by username
*/
public function getUserByUsername($username) {
$sql = "SELECT * FROM users WHERE username = ?";
return $this->db->fetchOne($sql, [$username]);
}
/**
* Get user by email
*/
public function getUserByEmail($email) {
$sql = "SELECT * FROM users WHERE email = ?";
return $this->db->fetchOne($sql, [$email]);
}
/**
* Create new user
*/
public function createUser($data) {
// Check if username exists
if ($this->getUserByUsername($data['username'])) {
return ['success' => false, 'message' => 'Username already exists'];
}
// Check if email exists
if ($this->getUserByEmail($data['email'])) {
return ['success' => false, 'message' => 'Email already exists'];
}
// Hash password
$passwordHash = password_hash($data['password'], PASSWORD_DEFAULT);
```

```
// Prepare user data
$userData = [
'username' => $data['username'],
'email' => $data['email'],
'password_hash' => $passwordHash,
'full_name' => $data['full_name'],
'phone' => $data['phone'] ?? null,
'role' => $data['role'],
'status' => $data['status'] ?? 'active',
'created_by' => $_SESSION['user_id'] ?? null
];
$this->db->beginTransaction();
try {
// Insert user
$userId = $this->db->insert('users', $userData);
if (!$userId) {
throw new Exception('Failed to create user');
}
// Set default permissions based on role
$permissions = $this->getDefaultPermissions($data['role']);
$permissions['user_id'] = $userId;
// Merge with custom permissions if provided
if (isset($data['permissions'])) {
$permissions = array_merge($permissions, $data['permissions']);
}
// Insert permissions
$this->db->insert('user_permissions', $permissions);
$this->db->commit();
// Log activity
$this->logActivity($userId, 'user_created', "User {$data['username']} created");
return ['success' => true, 'message' => 'User created successfully', 'user_id' => $userId];
} catch (Exception $e) {
$this->db->rollback();
return ['success' => false, 'message' => $e->getMessage()];
}
}
/**
* Update user
*/
public function updateUser($userId, $data) {
// Check if username exists (excluding current user)
$existingUser = $this->getUserByUsername($data['username']);
if ($existingUser && $existingUser['user_id'] != $userId) {
return ['success' => false, 'message' => 'Username already exists'];
}
// Check if email exists (excluding current user)
$existingEmail = $this->getUserByEmail($data['email']);
if ($existingEmail && $existingEmail['user_id'] != $userId) {
return ['success' => false, 'message' => 'Email already exists'];
}
```

```
// Prepare user data
$userData = [
'username' => $data['username'],
'email' => $data['email'],
'full_name' => $data['full_name'],
'phone' => $data['phone'] ?? null,
'role' => $data['role'],
'status' => $data['status']
];
// Update password if provided
if (!empty($data['password'])) {
$userData['password_hash'] = password_hash($data['password'], PASSWORD_DEFAULT);
}
$this->db->beginTransaction();
try {
// Update user
$updated = $this->db->update('users', $userData, 'user_id = ?', [$userId]);
if (!$updated) {
throw new Exception('Failed to update user');
}
// Update permissions if provided
if (isset($data['permissions'])) {
$permissions = $data['permissions'];
$permissions['user_id'] = $userId;
// Check if permissions exist
$existingPermissions = $this->db->fetchOne(
"SELECT * FROM user_permissions WHERE user_id = ?",
[$userId]
);
if ($existingPermissions) {
$this->db->update('user_permissions', $permissions, 'user_id = ?', [$userId]);
} else {
$this->db->insert('user_permissions', $permissions);
}
}
$this->db->commit();
// Log activity
$this->logActivity($userId, 'user_updated', "User {$data['username']} updated");
return ['success' => true, 'message' => 'User updated successfully'];
} catch (Exception $e) {
$this->db->rollback();
return ['success' => false, 'message' => $e->getMessage()];
}
}
/**
* Delete user (soft delete by setting status to inactive)
*/
public function deleteUser($userId) {
// Prevent deleting own account
if ($userId == $_SESSION['user_id']) {
```

```
return ['success' => false, 'message' => 'Cannot delete your own account'];
}
$updated = $this->db->update(
'users',
['status' => 'inactive'],
'user_id = ?',
[$userId]
);
if ($updated) {
$this->logActivity($userId, 'user_deleted', "User account deactivated");
return ['success' => true, 'message' => 'User deactivated successfully'];
}
return ['success' => false, 'message' => 'Failed to deactivate user'];
}
/**
* Permanently delete user (use with caution)
*/
public function permanentlyDeleteUser($userId) {
// Prevent deleting own account
if ($userId == $_SESSION['user_id']) {
return ['success' => false, 'message' => 'Cannot delete your own account'];
}
$this->db->beginTransaction();
try {
// Delete permissions first
$this->db->delete('user_permissions', 'user_id = ?', [$userId]);
// Delete user
$deleted = $this->db->delete('users', 'user_id = ?', [$userId]);
if (!$deleted) {
throw new Exception('Failed to delete user');
}
$this->db->commit();
return ['success' => true, 'message' => 'User deleted permanently'];
} catch (Exception $e) {
$this->db->rollback();
return ['success' => false, 'message' => $e->getMessage()];
}
}
/**
* Change user password
*/
public function changePassword($userId, $currentPassword, $newPassword) {
$user = $this->getUserById($userId);
if (!$user) {
return ['success' => false, 'message' => 'User not found'];
}
// Verify current password
if (!password_verify($currentPassword, $user['password_hash'])) {
return ['success' => false, 'message' => 'Current password is incorrect'];
}
```

```
// Hash new password
$passwordHash = password_hash($newPassword, PASSWORD_DEFAULT);
$updated = $this->db->update(
'users',
['password_hash' => $passwordHash],
'user_id = ?',
[$userId]
);
if ($updated) {
$this->logActivity($userId, 'password_changed', "Password changed");
return ['success' => true, 'message' => 'Password changed successfully'];
}
return ['success' => false, 'message' => 'Failed to change password'];
}
/**
* Reset user password (admin function)
*/
public function resetPassword($userId, $newPassword) {
$passwordHash = password_hash($newPassword, PASSWORD_DEFAULT);
$updated = $this->db->update(
'users',
['password_hash' => $passwordHash],
'user_id = ?',
[$userId]
);
if ($updated) {
$this->logActivity($userId, 'password_reset', "Password reset by admin");
return ['success' => true, 'message' => 'Password reset successfully'];
}
return ['success' => false, 'message' => 'Failed to reset password'];
}
/**
* Update user permissions
*/
public function updatePermissions($userId, $permissions) {
$permissions['user_id'] = $userId;
// Check if permissions exist
$existing = $this->db->fetchOne(
"SELECT * FROM user_permissions WHERE user_id = ?",
[$userId]
);
if ($existing) {
$updated = $this->db->update('user_permissions', $permissions, 'user_id = ?', [$userId]);
} else {
$updated = $this->db->insert('user_permissions', $permissions);
}
if ($updated) {
$this->logActivity($userId, 'permissions_updated', "User permissions updated");
return ['success' => true, 'message' => 'Permissions updated successfully'];
}
```

```
return ['success' => false, 'message' => 'Failed to update permissions'];
}
/**
* Get user permissions
*/
public function getUserPermissions($userId) {
$sql = "SELECT * FROM user_permissions WHERE user_id = ?";
return $this->db->fetchOne($sql, [$userId]);
}
/**
* Get default permissions based on role
*/
private function getDefaultPermissions($role) {
$permissions = [
'can_add_products' => false,
'can_edit_products' => false,
'can_delete_products' => false,
'can_manage_stock' => false,
'can_make_sales' => false,
'can_view_reports' => false,
'can_manage_users' => false,
'can_manage_customers' => false,
'can_process_refunds' => false,
'can_give_discounts' => false,
'max_discount_percent' => 0
];
switch ($role) {
case 'admin':
// Admin has all permissions
foreach ($permissions as $key => $value) {
if ($key === 'max_discount_percent') {
$permissions[$key] = 100;
} else {
$permissions[$key] = true;
}
}
break;
case 'manager':
$permissions['can_add_products'] = true;
$permissions['can_edit_products'] = true;
$permissions['can_manage_stock'] = true;
$permissions['can_make_sales'] = true;
$permissions['can_view_reports'] = true;
$permissions['can_manage_customers'] = true;
$permissions['can_process_refunds'] = true;
$permissions['can_give_discounts'] = true;
$permissions['max_discount_percent'] = 20;
break;
case 'cashier':
$permissions['can_make_sales'] = true;
```

```
$permissions['can_manage_customers'] = true;
$permissions['can_give_discounts'] = true;
$permissions['max_discount_percent'] = 5;
break;
}
return $permissions;
}
/**
* Get user statistics
*/
public function getUserStats($userId) {
$stats = [];
// Total sales
$sales = $this->db->fetchOne(
"SELECT COUNT(*) as total_sales, COALESCE(SUM(total_amount), 0) as total_amount
FROM sales WHERE served_by = ? AND sale_status = 'completed'",
[$userId]
);
$stats['total_sales'] = $sales['total_sales'];
$stats['total_amount'] = $sales['total_amount'];
// Today's sales
$todaySales = $this->db->fetchOne(
"SELECT COUNT(*) as count, COALESCE(SUM(total_amount), 0) as amount
FROM sales WHERE served_by = ? AND DATE(sale_date) = CURDATE()
AND sale_status = 'completed'",
[$userId]
);
$stats['today_sales'] = $todaySales['count'];
$stats['today_amount'] = $todaySales['amount'];
// Last login
$user = $this->getUserById($userId);
$stats['last_login'] = $user['last_login'];
return $stats;
}
/**
* Get active cashiers
*/
public function getActiveCashiers() {
$sql = "SELECT user_id, username, full_name
FROM users
WHERE role IN ('cashier', 'manager', 'admin')
AND status = 'active'
ORDER BY full_name";
return $this->db->fetchAll($sql);
}
/**
* Log user activity
*/
private function logActivity($userId, $type, $description) {
$data = [
```

```
'user_id' => $userId,
'activity_type' => $type,
'activity_description' => $description,
'ip_address' => $_SERVER['REMOTE_ADDR'] ?? 'Unknown',
'user_agent' => $_SERVER['HTTP_USER_AGENT'] ?? 'Unknown'
];
$this->db->insert('activity_log', $data);
}
/**
* Get user activity log
*/
public function getUserActivityLog($userId, $limit = 50) {
$sql = "SELECT * FROM activity_log
WHERE user_id = ?
ORDER BY created_at DESC
LIMIT ?";
return $this->db->fetchAll($sql, [$userId, $limit]);
}
/**
* Count users by role
*/
public function countUsersByRole() {
$sql = "SELECT role, COUNT(*) as count
FROM users
WHERE status = 'active'
GROUP BY role";
return $this->db->fetchAll($sql);
}
/**
* Search users
*/
public function searchUsers($searchTerm) {
$sql = "SELECT * FROM users
WHERE (username LIKE ? OR full_name LIKE ? OR email LIKE ?)
AND status = 'active'
ORDER BY full_name";
$term = "%{$searchTerm}%";
return $this->db->fetchAll($sql, [$term, $term, $term]);
}
}
?>
```